INDEX

DSA

1: Array-Based Programs

1. Insert an element at a given position.
2. Delete an element from a given position.
3. Search for an element using linear search.
4. Search for an element using binary search.
5. Reverse an array (in-place).
6. Merge two arrays.
7. Remove duplicate elements from an array.
8. Implement matrix multiplication.

2: Singly Linked List Programs

9. Create a singly linked list and count the number of nodes.
10. Insert a node at beginning, end, and specific position.
11. Reverse a singly linked list.
12. Merge two singly linked lists.

3: Tree Programs

13. Binary Tree Traversal.

1. Insert an element at a given position.

```
#include <iostream>
using namespace std;

void insert(int item, int index, int *array, int size)
{
    int i = size - 2;
    // Right shifting
    while (i >= index)
    {
        array[i + 1] = array[i];
        i--;
    }
    array[index] = item;
}
int main()
{
    int array[] = {1, 2, 3, 4, 5};
    insert(2, 3, array, 5);
    for (int i : array)
    {
        cout << i << " , ";
    }
    return 0;
}
```

OUTPUT :

```
1 , 2 , 3 , 2 , 4 ,
```

2. Delete an element from a given position.

```
#include <iostream>
using namespace std;
void remove(int index, int *array, int size)
{
    // Left Shifting
    for (int i = index; i < size; i++)
    {
        array[i] = array[i + 1];
    }
    array[size - 1] = 0;
}

int main()
{
    int array[] = {1, 2, 3, 4, 5, 6, 7};
    remove(3, array, 7);
    for (int i : array)
    {
        cout << i << " , ";
    }

    return 0;
}
```

OUTPUT :

```
1 , 2 , 3 , 5 , 6 , 7 , 0 ,
```

3. Search for an element using linear search.

```
#include <iostream>
using namespace std;
int search(int item, int *array, int size)
{
    // Left Shifting
    for (int i = 0; i < size; i++)
    {
        if (array[i] == item)
            return i;
    }
    return -1;
}

int main()
{
    int array[] = {1, 2, 3, 4, 5, 6, 7};
    cout << "Found 7 at :  " << search(7, array, 7) << "\n";
    cout << "Found 8 at : " << search(8, array, 7);
    return 0;
}
```

OUTPUT :

```
Found 7 at :  6
Found 8 at : -1
```

4. Search for an element using binary search.

```
#include <iostream>
using namespace std;
int search(int item, int low, int high, int *array, int
size)
{
    while (low <= high)
    {
        int mid = high - (high - low) / 2; // to avoid
overflow
        int current = array[mid];
        if (item < current)
            return search(item, low, mid - 1, array, size);
        else if (item > current)
            return search(item, mid + 1, high, array, size);
        else
            return mid;
    }
    return -1;
}
int main()
{
    int array[] = {1, 2, 3, 4, 5, 6, 7};
    cout << "Found 2 at : " << search(2, 0, 6, array, 7) <<
"\n";
    cout << "Found 6 at : " << search(6, 0, 6, array, 7) <<
"\n";
    cout << "Found 8 at : " << search(8, 0, 6, array, 7);
    return 0;
}
```

OUTPUT :

```
Found 2 at : 1
Found 6 at : 5
Found 8 at : -1
```

5. Reverse an array (in-place).

```
#include <iostream>
using namespace std;
void reverse(int *array, int size)
{
    int low = 0;
    int high = size - 1;
    while (low <= high)
    {
        swap(array[low], array[high]);
        low++;
        high--;
    }
}
int main()
{
    int array[] = {1, 2, 3, 4, 5, 6, 7};
    reverse(array, 7);
    for (int i : array)
    {
        cout << i << " , ";
    }
    return 0;
}
```

OUTPUT :

```
7 , 6 , 5 , 4 , 3 , 2 , 1 ,
```

6. Merge two arrays.

```
#include <iostream>
using namespace std;
int *merge(int *array1, int size1, int *array2, int size2)
{
    int total_size = size1 + size2;
    int *result = new int[total_size];
    int i = 0;
    for (; i < size1; i++)
    {
        result[i] = array1[i];
    }
    for (; i < total_size; i++)
    {
        result[i] = array2[i - size1];
    }
    return result;
}
int main()
{
    int array1[] = {1, 2, 3, 4, 5, 6, 7};
    int array2[] = {1, 22, 13, 4};
    int *array3 = merge(array1, 7, array2, 4);
    for (int i = 0; i < 11; i++)
    {
        cout << i[array3] << " , ";
    }
    return 0;
}
```

OUTPUT :

```
1 , 2 , 3 , 4 , 5 , 6 , 7 , 1 , 22 , 13 , 4 ,
```

7. Remove duplicate elements from an array.

```
#include <iostream>
using namespace std;
void remove_duplicate(int *array, int size)
{
    int *result = new int[size];
    int k = 0;
    for (int i = 0; i < size; i++)
    {   bool found = false; int current = array[i];
        for (int j = k; j >= 0; j--)
        {if (current == result[j])
              {found = true; break ;}
        }
        if (!found) {result[k] = current; k++;}
    for (int i = 0; i < k; i++)
    {
        cout << result[i] << " , ";
    }
}
int main()
{
    int array[] = {1, 2, 1, 4, 4, 6, 7, 1, 0, 7, 9, 6};
    for (int i : array)
    {
        cout << i << " , ";
    }
    cout << endl;
    remove_duplicate(array, 12);
    return 0;
}
```

OUTPUT :

```
1 , 2 , 1 , 4 , 4 , 6 , 7 , 1 , 0 , 7 , 9 , 6 ,
1 , 2 , 4 , 6 , 7 , 0 , 9 ,
```

8. Implement matrix multiplication.

```
#include <iostream>
using namespace std;
void matix_multiplication(int m1[][4], int m2[][2], int r1,
int c1, int r2, int c2)
{
    if (c1 != r2)
    {
        cout << "row in m1 must be equal to col in m2";
        return;
    }
    int **result = new int *[r1];
    for (int i = 0; i < r1; i++)
    {
        result[i] = new int[c2];
    }
    for (int i = 0; i < r1; i++)
    {
        for (int j = 0; j < c2; j++)
        {
            int sum = 0;
            for (int k = 0; k < c1; k++)
            {
                sum += m1[i][k] * m2[k][j];
            }
            result[i][j] = sum;
        }
    }
    int r = r1;
    int c = c2;
    cout << "\n";
    for (int i = 0; i < r; i++)
    {
        for (int j = 0; j < c; j++)
        {
            cout << result[i][j] << ",";
        }
        cout << "\n";
```

```
    }
    cout << "\n";
}

int main()
{
    int m1[2][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}};
    int m2[4][2] = {{1, 2}, {3, 4}, {5, 6}, {7, 8}};
    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 4; j++)
        {cout << m1[i][j] << ",";
        }cout << "\n";
    }
    cout << "\n";
    for (int i = 0; i < 4; i++)
    {
        for (int j = 0; j < 2; j++)
        {cout << m2[i][j] << ",";
        }cout << "\n";
    }
    matix_multiplication(m1, m2, 2, 4, 4, 2);
    return 0;
}
```

OUTPUT :

```
1,2,3,4,
5,6,7,8,

1,2,
3,4,
5,6,
7,8,

50,60,
114,140,
```

9. Create a singly linked list and count the number of nodes.

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *node;
};

int count(Node *head)
{
    int number_of_nodes = 0;
    while (head)
    {
        head = head->node;
        number_of_nodes++;
    }
    return number_of_nodes;
}
int main()
{
    Node *n3 = new Node{3, NULL};
    Node *n2 = new Node{3, n3};
    Node *n1 = new Node{3, n2};

    cout << "size : " << count(n1);
    return 0;
}
```

OUTPUT :

```
size : 3
```

10. Insert a node at beginning, end, and specific position.

```
#include <iostream>
using namespace std;
struct Node
{
    int data;
    Node *next;
};
Node *head;
void print()
{
    Node *temp = head;
    while (temp)
    {
        cout << temp->data << " -->";
        temp = temp->next;
    }
}
void insert_beginning(int value)
{
    Node *new_node = new Node{value, NULL};
    new_node->next = head;
    head = new_node;
}
void insert_end(int value)
{
    Node *new_node = new Node{value, NULL};
    if (!head)
    {
        head = new_node;
        return;
    }
    Node *temp = head;
    while (temp->next)
    {
        temp = temp->next;
```

```
    }
    temp->next = new_node;
}
void insert_location(int value, int index)
{
    if (!head || index == 0)
    {
        insert_beginning(value);
        return;
    }

    Node *new_node = new Node{value, NULL};
    Node *temp = head;
    for (int i = 0; i < index - 1 && temp->next; i++)
    {
        temp = temp->next;
    }
    new_node->next = temp->next;
    temp->next = new_node;
}
int main()
{
    insert_beginning(2);
    insert_beginning(1);
    insert_end(3);
    insert_end(4);
    insert_location(-1, 0);
    insert_location(5, 2);
    print();
    return 0;
}
```

OUTPUT :

```
-1 -->1 -->5 -->2 -->3 -->4 -->
```

11. Reverse a singly linked list.

```
#include <iostream>
using namespace std;
struct Node
{
    int data;
    Node *next;
};
Node *head;

void print()
{
    Node *temp = head;
    while (temp)
    {
        cout << temp->data << " -->";
        temp = temp->next;
    }
    cout << "\n";
}

void reverse()
{
    Node *prev = NULL, *next = NULL, *cur = head;
    while (cur)
    {
        next = cur->next;
        cur->next = prev;
        prev = cur;
        cur = next;
    }
    head = prev;
}

int main()
{
```

```
    Node *n3 = new Node{3, NULL};
    Node *n2 = new Node{2, n3};
    head = new Node{1, n2};
    print();
    reverse();
    print();
    return 0;
}
```

OUTPUT :

```
1 -->2 -->3 -->
3 -->2 -->1 -->

c:\Users\danny\OneDrive\Desktop
```

12. Merge two singly linked lists.

```
#include <iostream>
using namespace std;
#define N(x, y) \
    new Node { x, y }
struct Node
{
    int data;
    Node *next;
};

void print(Node *head)
{
    Node *temp = head;
    while (temp)
    {
        cout << temp->data << " -->";
        temp = temp->next;
    }
    cout << "\n";
}
Node *merge(Node *l1, Node *l2)
{
    if (!l1)
        return l2;
    Node *temp = l1;
    while (temp->next)
    {
        temp = temp->next;
    }
    temp->next = l2;
    return l1;
}

int main()
{
```

```
    Node *l1, *l2;
    l1 = N(1, N(2, N(3, NULL)));
    l2 = N(4, N(5, N(6, NULL)));
    print(l1);
    print(l2);
    merge(l1, l2);
    print(l1);
    return 0;
}
```

OUTPUT :

```
1 -->2 -->3 -->
4 -->5 -->6 -->
1 -->2 -->3 -->4 -->5 -->6 -->
```

13. Binary Tree Traversal.

```
#include <iostream>
#include <stack>
using namespace std;
struct Node
{
    int data;
    Node *left = NULL, *right = NULL;
};
void printNode(Node *node, int indentation)
{
    if (!node)
        return;

    string indent(indentation, ' ');
    cout << indent << "->" << node->data << "\n";

    printNode(node->left, indentation + 2);
    printNode(node->right, indentation + 2);
}
void printTree(Node *root)
{
    cout << "Tree\n";
    printNode(root, 0);
}

void print(Node *node)
{
    cout << "[" << node->data << "]";
}
void preOrder(Node *root)
{
    if (root == NULL)
        return;
    stack<Node *> s;
    Node *cur = root;

    while (cur)
    {
        print(cur);
        if (cur->right)
            s.push(cur->right);
        if (cur->left)
            cur = cur->left;
        else
        {
            cur = s.top();
```

```
            s.pop();
        }
    }
}

int main()
{
    Node *n5 = new Node{5};
    Node *n4 = new Node{4};
    Node *n3 = new Node{3};
    Node *n2 = new Node{2};
    Node *n1 = new Node{1};
    Node *root = new Node{0};

    root->left = n1;
    root->right = n2;
    n1->left = n3;
    n1->right = n4;
    n2->left = n5;

    printTree(root);
    cout << "Preorder Traversal : ";
    preOrder(root);
    return 0;
}
```

OUTPUT :

```
Tree
->0
  ->1
    ->3
    ->4
  ->2
    ->5
Preorder Traversal : [0][1][3][4][2][5]
```